

Chapter 2 NoSQL Data Models

1. Aggregate Data Model in NoSQL

The Aggregate Data Model is a **NoSQL** data model that groups related data into a single unit called an aggregate. It stores related information together, reducing the need for complex joins and improving data access efficiency.

- Stores related data as a single aggregate, commonly in JSON or BSON documents.
- Implemented in MongoDB, CouchDB and Redis for aggregate-based storage.
- Reduces cross-aggregate dependencies during data operations.

1.1 Components of Aggregate Data Model

- **Aggregate Root:** The main entity that manages and controls the aggregate.
- **Entities:** Objects within the aggregate that have unique identities.
- **Value Objects:** Objects that store descriptive information without a unique identity.
- **Embedded Data:** Related data stored directly inside the aggregate.
- **References:** Links to other aggregates or external data when needed.

Example: A customer document may contain a Membership ID that refers to a separate membership aggregate containing membership details.

```
{
  "customerName": "John Smith",
  "city": "New York",
  "orders": [
    {
      "product": "Laptop",
      "quantity": 1
    },
    {
      "product": "Mouse",
      "quantity": 2
    }
  ]
}
```

Step 1: Group Related Data Together

Related information is combined into a single aggregate instead of being stored in multiple tables.

Example:

```
{
  "customerName": "John Smith",
  "orders": [
    {
      "product": "Laptop",
      "quantity": 1
    }
  ]
}
```

Step 2: Store the Aggregate as a Single Document

The entire aggregate is stored in one document within a collection.

Example:

```
db.customers.insertOne({
  "customerName": "John Smith",
  "orders": [
    {
      "product": "Laptop",
      "quantity": 1
    }
  ]
})
```

Step 3: Retrieve the Aggregate

The complete aggregate can be fetched with a single query.

Example:

```
db.customers.find({
  "customerName": "John Smith"
})
```

Step 4: Update the Aggregate

Changes to related data can be made directly within the aggregate.

Example:

```
db.customers.updateOne(
  { "customerName": "John Smith" },
  { $set: { "city": "Chicago" } }
)
```

Step 5: Return Updated Data

After the update, the aggregate contains the latest information.

Example:

```
{
  "customerName": "John Smith",
  "city": "Chicago",
  "orders": [
    {
      "product": "Laptop",
      "quantity": 1
    }
  ]
}
```

The updated aggregate now includes the customer's new city while keeping all related order information together in a single document.

1.2 Aggregate-Oriented NoSQL Data Models

Aggregate-Oriented NoSQL Data Models are classified into the following types:

- **Key-Value Data Model**
- **Document Data Model**
- **Column-Family Data Model**

1.3 Aggregate Data Model Example in MongoDB

MongoDB is a document-oriented NoSQL database that stores related information together in a single document. Such a document acts as an aggregate because all data associated with a business entity is grouped into one unit.

Example

```
{
  "_id": "ORD1001",
  "customer": {
    "customerId": "C101",
    "name": "John Doe",
    "email": "john@example.com"
  },
  "items": [
    {
      "productId": "P101",
      "productName": "Laptop",
      "quantity": 1,
      "price": 50000
    },
    {
      "productId": "P102",
      "productName": "Mouse",
      "quantity": 2,
      "price": 1000
    }
  ],
  "totalAmount": 52000
}
```

- `_id` uniquely identifies the order.
- The `customer` field stores customer information.
- The `items` array contains details of all products included in the order.
- `totalAmount` stores the total value of the order.

1.4 Advantages of Aggregate Data Model

- **Faster Data Access:** Related data is stored in a single aggregate.
- **Efficient Query Processing:** Reduces the need for multiple database operations.
- **Easy Horizontal Scaling:** Suitable for distributed NoSQL systems.
- **Supports Atomic Updates:** Changes within an aggregate can be updated as a single unit.

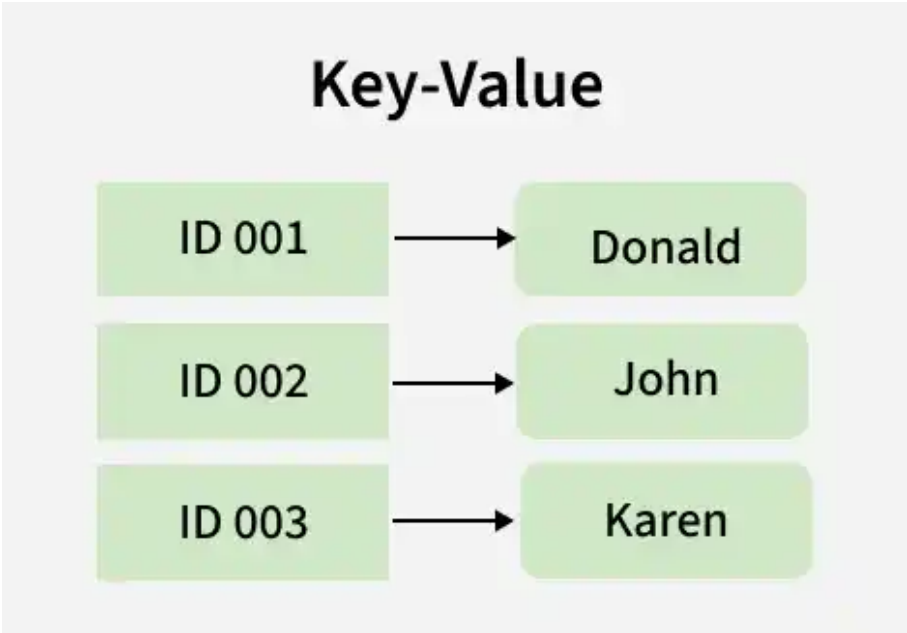
1.5 Limitations of Aggregate Data Model

- **Data Redundancy:** Same data may be stored in multiple aggregates.
- **Update Complexity:** Updating duplicated data is more difficult.
- **Limited Relationships:** Not ideal for complex many-to-many relationships.

2. Key Value Data Model

A key-value database is a type of NoSQL database that stores data as key-value pairs. Each key acts as a unique identifier and is associated with a single value.

- Uses simple data storage architecture.
- Handles large volumes of distributed data efficiently.
- Commonly used in high-performance applications.



2.1 Components of a Key-Value Data Model

The Key-Value Data Model consists of the following components:

2.1.1 Key

A unique identifier used to access a specific piece of data. The database uses the key to quickly find the associated value without scanning all records.

Example:

```
Key: User101
```

2.1.2 Value

The data associated with the key, which can be a string, number, JSON object, image or any other data type. The structure of the value is flexible, allowing applications to store data in various formats based on their requirements.

Example:

```
{
  "Name": "John",
  "Age": 25,
  "City": "Delhi"
}
```

2.1.3 Key-Value Pair

The combination of a key and its corresponding value. Data is inserted, updated and retrieved using these key-value pairs.

Example:

Key	Value
User101	{"Name":"John","Age":25,"City":"Delhi"}
User102	{"Name":"Emma","Age":30,"City":"Mumbai"}

2.2 Working of a Key-Value Database

The working of a key-value database is based on storing and retrieving data using unique keys. Each key is associated with a specific value, allowing fast and efficient data access.

Step 1: Store Data as Key-Value Pairs

The database stores data in the form of key-value pairs, where each key acts as a unique identifier for its corresponding value.

Example:

Key	Value
101	John
102	Alice
103	Bob

Step 2: Request Data Using a Key

When a user wants to retrieve data, they provide the key associated with the required value.

Example:

```
Search for key102.
```

Step 3: Locate the Key

The database uses its indexing mechanism to quickly locate the specified key without scanning the entire dataset.

Example:

```
When the user searches for key102, the database quickly locates the corresponding key-value pair in its storage.
```

Step 4: Return the Associated Value

Once the key is found, the database returns the corresponding value.

Output:

```
Alice
```

Step 5: Update or Delete Data

The value associated with a key can be updated or the key-value pair can be removed whenever required.

Example (Update):

If the value associated with key 102 changes from Alice to Emma, the database updates the existing value while keeping the same key.

Key	Old Value	New Value
102	Alice	Emma

Example (Delete):

If the data associated with key 103 is no longer required, the database removes the entire key-value pair.

Key	Value
103	Bob

After deletion, key 103 and its value are no longer available in the database.

2.3 Uses of Key-Value Model

Key-value databases are widely used in applications that require fast data storage and retrieval.

2.3.1 Caching

Key-value databases are used to store frequently accessed data in memory, reducing the need to repeatedly query the main database.

Example:

```
Key: Product_101
Value: Laptop Details
```

2.3.2 Session Management

Web applications store user session information as key-value pairs to maintain login states and user preferences.

Example:

```
Key: Session_12345
Value: User Logged In
```

The application uses the session ID to identify the user during their visit.

2.3.3 User Profiles

User information can be stored and retrieved using a unique user ID as the key.

Example:

```
Key: User_101
Value: {
  "Name": "John",
  "Email": "john@example.com"
}
```

2.3.4 Shopping Carts

E-commerce websites use key-value databases to store products added to a user's shopping cart.

Example:

```
Key: Cart_101
Value: ["Laptop", "Mouse", "Keyboard"]
```

This allows quick access to cart details during checkout.

2.3.5 Configuration Management

Application settings and configuration data are often stored as key-value pairs.

Example:

Key	Value
Theme	Dark Mode
Language	English
TimeZone	UTC+5:30

Applications can quickly retrieve these settings when they start.

2.3.6 IoT Applications

IoT systems store device information and sensor readings using unique device IDs.

Example:

```
Key: Sensor_001
Value: Temperature = 28°C
```

The latest sensor data can be accessed instantly using the device key.

2.4 Example of Key-Value Data Model

The following example stores user information using a key-value structure:

```
{
  "User101": {
    "Name": "John",
    "Age": 25,
    "City": "Delhi"
  },
}
```

```

    "User102": {
      "Name": "Alice",
      "Age": 30,
      "City": "Mumbai"
    },
    "User103": {
      "Name": "Bob",
      "Age": 28,
      "City": "Bangalore"
    }
  }
}

```

- User101, User102 and User103 are the keys.
- The corresponding user details are the values.
- Data is retrieved by specifying the required key, such as User102, which returns Alice's information.

2.5 Databases

- **Couchbase:** Supports SQL-like querying and text search.
- **Amazon DynamoDB:** Highly scalable and widely used cloud-based key-value database.
- **Riak:** Designed for building distributed applications.
- **Aerospike:** Open-source database optimized for real-time applications.
- **Berkeley DB:** High-performance database with scalability support.

2.6 Advantages of Key Value Model

- **Efficient Key-Based Access:** Enables rapid data retrieval through unique keys.
- **Low-Latency Operations:** Provides high-performance read and write operations.
- **Horizontal Scalability:** Supports data distribution across multiple nodes.
- **Schema Flexibility:** Allows storage of diverse data types without a predefined schema.
- **High Availability:** Ensures continuous access to data in distributed environments.

2.7 Limitations of Key Value Model

- **Limited Query Capabilities:** Efficiently supports only key-based lookups, making complex queries difficult.
- **No Relationship Support:** Does not provide native support for relationships between data entities.
- **Data Redundancy:** May require duplication of related data across multiple key-value pairs.

3. Document Databases in NoSQL

The Document Database Model is a type of **NoSQL** database model that stores data in the form of documents instead of rows and columns. Documents are usually stored in formats such as JSON, BSON or XML, making them flexible and easy to manage.

- Data is stored as documents rather than rows and tables.
- Supports flexible and dynamic schemas.
- Documents can contain nested data and complex structures.
- Designed for high scalability and fast data retrieval.

3.1 Components of a Document Data Model

A Document Data Model consists of components that store and organize data as documents, enabling flexible and hierarchical data management.

3.1.1 Document

A document is the basic unit of data storage. It contains related information organized as key-value pairs.

Example:

```
{
  "_id": 101,
  "name": "John",
  "age": 25,
  "city": "Delhi"
}
```

3.1.2 Fields

Fields are individual attributes within a document. Each field consists of a field name and a corresponding value.

Example:

Field	Value
name	John
age	25
city	Delhi

3.1.3 Collection

A collection is a group of related documents. It is similar to a table in a relational database but does not require a fixed schema.

Example:

Users Collection

```
[
  {
    "_id": 101,
    "name": "John",
    "age": 25
  },
  {
    "_id": 102,
    "name": "Alice",
    "age": 30
  }
]
```

3.1.4 Embedded Documents

A document can contain another document within it, allowing related data to be stored together.

Example:

```
{
  "_id": 101,
  "name": "John",
  "address": {
    "city": "Delhi",
    "pincode": 110001
  }
}
```

3.2 Working of Document Database Model

A Document Database Model stores data as documents within collections, enabling flexible storage and retrieval of semi-structured data.

Step 1: Store Data as Documents

Instead of storing data in rows, a document database stores information in individual documents.

Example:

```
{
  "StudentID": 101,
  "Name": "David",
  "Branch": "Computer",
  "Age": 20
}
```

Step 2: Group Documents into Collections

Similar documents are stored together in a collection.

Students Collection:

```
{
  "StudentID": 101,
  "Name": "David",
  "Branch": "Computer"
}

{
  "StudentID": 102,
  "Name": "Alen",
  "Branch": "Electronics"
}
```

Step 3: Insert Data

New documents can be added without modifying the existing structure of the collection.

Step 4: Retrieve Data

The database searches and retrieves documents directly based on fields such as StudentID, Name or Branch.

Step 5: Scale Across Servers

Document databases can distribute collections across multiple servers to handle large volumes of data efficiently.

3.3 Example of a Document Database in NoSQL

The following example illustrates an employee document stored in a NoSQL database.

Employee Document (JSON Format):

```
{
  "employee_id": 101,
  "name": "John Smith",
  "department": "IT",
  "email": "john.smith@example.com",
  "address": {
    "city": "New York",
    "country": "USA"
  },
  "projects": [
    {
      "project_id": 1,
      "project_name": "Website Development"
    }
  ]
}
```

```

    },
    {
      "project_id": 2,
      "project_name": "Mobile App"
    }
  ]
}

```

- Data is stored as a document.
- Employee details, address and projects are stored together.
- Flexible schema allows adding new fields easily.
- All related data can be retrieved with a single query.

3.4 Popular Document Databases

- MongoDB
- CouchDB
- Amazon DocumentDB
- Azure Cosmos DB

3.5 Advantages of Document Database Model

- **Flexible Schema:** Documents can have different fields and structures without affecting other documents.
- **Easy Data Representation:** Complex and nested data can be stored naturally within a single document.
- **High Scalability:** Data can be distributed across multiple servers to support large-scale applications.
- **Fast Data Access:** Documents can be retrieved quickly without requiring complex joins.
- **Developer Friendly:** JSON-like document formats are easy to understand and work with in modern applications.

3.6 Limitations of Document Database Model

- **Data Redundancy:** Similar information may be repeated across multiple documents, increasing storage usage.
- **Complex Relationships:** Managing relationships between documents can be more difficult than in relational databases.
- **Limited Join Support:** Most document databases provide limited or less efficient join operations.

4. JSON Databases in NoSQL

JSON databases store data in JSON documents, providing a flexible and schema-free approach for managing semi-structured and unstructured data.

- Supports dynamic document structures with varying field values.
- Uses MongoDB, CouchDB and Azure Cosmos DB for JSON-based storage.
- Enables indexing of document fields for faster query execution.

4.1 Components of JSON Databases

Example:

```

{
  "id": 101,
  "name": "Alice",
  "department": "IT",
  "email": "alice@example.com",
  "address": {
    "city": "Delhi",

```

```
    "state": "Delhi"
  },
  "skills": ["Python", "SQL", "MongoDB"]
}
```

4.1.1 Document

The entire JSON object is a document that stores all information about one employee.

Example:

The complete JSON shown above is one employee document.

4.1.2 Collection

A collection stores multiple related documents.

Example:

The above document is stored in the Employees collection along with other employee documents.

4.1.3 Fields (Key-Value Pairs)

Each attribute inside the document is a field.

Example:

- id : 101
- name : "Alice"
- department : "IT"
- email : "alice@example.com"

4.1.4 Nested Objects and Arrays

Documents can contain nested objects and arrays.

Example:

- address is a nested object containing city and state.
- skills is an array containing "Python", "SQL" and "MongoDB".

4.1.5 Indexes

Suppose an index is created on the email field of this document.

Example:

```
db.employees.createIndex({ email: 1 })
```

Now, searching for "alice@example.com" becomes much faster because the database uses the index instead of scanning every document.

4.1.6 Query Engine

The query engine retrieves documents matching specified conditions.

Example:

```
db.employees.find({ department: "IT" })
```

The query engine returns the above document because its department field is "IT".

4.1.7 Storage Engine

When this document is inserted:

```
db.employees.insertOne({
  id: 101,
  name: "Alice",
  department: "IT",
  email: "alice@example.com"
})
```

The storage engine writes the document to storage, manages memory and indexing and retrieves it efficiently when queried.

4.2 Ways to Store JSON Data in JSON Database

There are two common ways to store JSON data in a JSON database:

4.2.1 Store JSON as a Document

The entire JSON object is stored as a single document. Each document contains all the related information for a particular entity.

Example:

```
{
  "employee_id": 101,
  "name": "John",
  "department": "IT"
}
```

4.2.2 Store JSON as Embedded Documents

Related data is stored as nested objects or arrays within the same JSON document. This approach keeps associated information together and reduces the need for multiple queries.

Example:

```
{
  "employee_id": 101,
  "name": "John",
  "department": "IT",
  "address": {
    "city": "New York",
    "country": "USA"
  },
  "skills": [
    "Python",
    "SQL"
  ]
}
```

4.3 Working of JSON Databases

The working of a JSON-based database can be understood through the four basic CRUD (Create, Read, Update, Delete) operations.

Step 1: Create a JSON Document

The first step is to create a JSON document containing the data to be stored. Each document consists of key-value pairs and may include nested objects or arrays.

Example:

```
{
  "employee_id": 101,
  "name": "John",
  "department": "HR"
}
```

Step 2: Store the Document

The JSON document is inserted into a collection. A collection groups similar documents together, similar to a table in a relational database.

Example (MongoDB):

```
db.employees.insertOne({
  "employee_id": 101,
  "name": "John",
  "department": "HR"
})
```

The document is stored in the employees collection.

Step 3: Retrieve the Document

The database retrieves the required document by executing a query on one or more fields.

Example (MongoDB):

```
db.employees.find({
  "employee_id": 101
})
```

Output:

```
{
  "employee_id": 101,
  "name": "John",
  "department": "HR"
}
```

Step 4: Update the Document

Individual fields in a JSON document can be modified without changing the entire document.

Example (MongoDB):

```
db.employees.updateOne(
  { "employee_id": 101 },
  {
    $set: {
      "salary": 75000
    }
  }
)
```

Updated Document:

```
{
  "employee_id": 101,
  "name": "John",
  "department": "HR",
  "salary": 75000
}
```

Step 5: Delete the Document

If a document is no longer required, it can be removed from the collection.

Example (MongoDB):

```
db.employees.deleteOne({
  "employee_id": 101
})
```

The document with employee_id = 101 is deleted from the employees collection.

4.4 Popular JSON Databases

Several NoSQL databases use JSON documents for data storage.

- **MongoDB:** Stores data as BSON (Binary JSON) documents.
- **CouchDB:** Provides JSON document storage with replication.
- **Couchbase:** Offers distributed and high-performance JSON storage.
- **Amazon DocumentDB:** A managed document database compatible with MongoDB.
- **Azure Cosmos DB:** Supports globally distributed JSON document storage.

4.5 Example of JSON Database

Consider a student management system.

JSON Database Approach

Consider an online bookstore, where each book is stored as a JSON document.

```
{
  "book_id": "B101",
  "title": "Database Systems",
  "author": {
    "name": "John Smith",
    "country": "USA"
  },
  "category": "Computer Science",
  "price": 799,
  "stock": 120,
  "formats": ["Paperback", "eBook"]
}
```

All book-related information is stored within a single JSON document, allowing faster retrieval.

4.6 Use Cases of JSON Databases

- **Artificial Intelligence (AI):** Store training datasets, model metadata and inference results.
- **Machine Learning (ML):** Manage feature data, model parameters and prediction outputs.
- **Microservices Architecture:** Exchange and store JSON documents between independent services.
- **REST APIs:** Process and store JSON request and response payloads efficiently.
- **Internet of Things (IoT):** Store sensor data, device configurations and telemetry information.

4.7 Advantages of JSON Databases

- **Flexible Schema:** Documents can have different structures.
- **Faster Development:** No schema migrations are required.
- **Better Performance:** Related data is stored together, reducing joins.

- **Easy Data Representation:** JSON matches objects used in programming languages.

4.8 Limitations of JSON Databases

- **Data Duplication:** Data may be repeated across documents.
- **Large Documents:** Large documents can slow down queries.
- **Limited Joins:** Complex relationships are difficult to manage.

5. Column Oriented Database

The Columnar Data Model is a type of **NoSQL** database model that stores data in columns instead of rows. It is designed to handle large amounts of data efficiently and provides high scalability.

- Designed to support scalable data storage across multiple nodes.
- Works with big data platforms using technologies such as Apache Cassandra and Apache HBase.
- Supports horizontal scaling while maintaining high availability across distributed systems.

5.1 Components of Columnar Data Model

The Columnar Data Model consists of several components that organize and manage data efficiently.

5.1.1 Row Key

The row key is a unique identifier for each record. It is used to locate and retrieve data efficiently.

Example:

```
EMP101
```

5.1.2 Column Family

A column family is a collection of related columns that are stored together.

Example:

```
Personal_Info
```

5.1.3 Columns

Columns store individual attributes such as Name, Age, Department or Salary.

Example:

```
Name : John
Age  : 30
```

5.1.4 Cell

A cell stores the value of a column for a particular row.

Example:

```
Salary : 70000
```

5.1.5 Timestamp

Many columnar databases associate a timestamp with each cell to maintain multiple versions of the data.

5.2 Example of Columnar Data Model

The following example shows how data is organized in a columnar data model.

Row Key	Personal_Info:Name	Personal_Info:Age	Academic_Info:Course
101	Ethan	21	Data Science
102	Harper	20	Computer Science

- **Row Key:** 101, 102
- **Column Families:** Personal_Info, Academic_Info
- **Columns:** Name, Age, Course
- **Values:** Ethan, 21, Data Science, etc.

This is the approach used by databases such as Apache Cassandra and Apache HBase.

5.3 Working of Columnar Data Model

Step 1: Store Data in Columns

Each record is stored using a unique row key, which allows the database to quickly locate the required data.

Row Key	Name	Course
101	Ethan	Data Science
102	Harper	Computer Science
103	Mason	Mathematics

Step 2: Organize Data by Columns

Related columns are grouped into column families instead of storing all columns in a single table

```
Row Key: 101

Personal_Info
  Name = Ethan

Academic_Info
  Course = Data Science
```

Step 3: Insert Data

When a new record is inserted, its values are stored under the appropriate row key and column families.

Example:

Row Key	Name	Course
104	Avery	Statistics

The new values are added to the corresponding column families.

Step 4: Retrieve Required Columns

Applications can retrieve only the required columns instead of unnecessary data, improving query efficiency.

Example:

```
SELECT Name
FROM Students
WHERE RowKey = 101;
```

Only the requested column (Name) is retrieved for the specified row.

Step 5: Scale for Large Datasets

Columnar databases distribute data across multiple servers, enabling high scalability and availability for large datasets.

Example:

A university stores millions of student records. The columnar database distributes data across multiple servers for faster storage and retrieval.

5.4 Columnar Data Model Databases

Some popular databases that use the Columnar Data Model are:

- Apache Cassandra
- Apache HBase
- Google Bigtable
- ScyllaDB

5.5 Advantages of Columnar Data Model

- **Well-Structured Storage:** Data is stored in a highly organized column-wise format, improving storage efficiency.
- **High Scalability:** Data can be distributed across multiple servers to handle large-scale workloads.
- **Fast Data Loading:** Large volumes of data can be loaded and processed efficiently

5.6 Limitations of Columnar Data Model

- **Complex Data Modeling:** Designing an efficient schema requires careful planning based on query patterns.
- **Limited Query Support:** Complex joins and aggregations are not as efficient as in relational databases.
- **Data Redundancy:** Data is often duplicated to improve query performance, increasing storage usage.

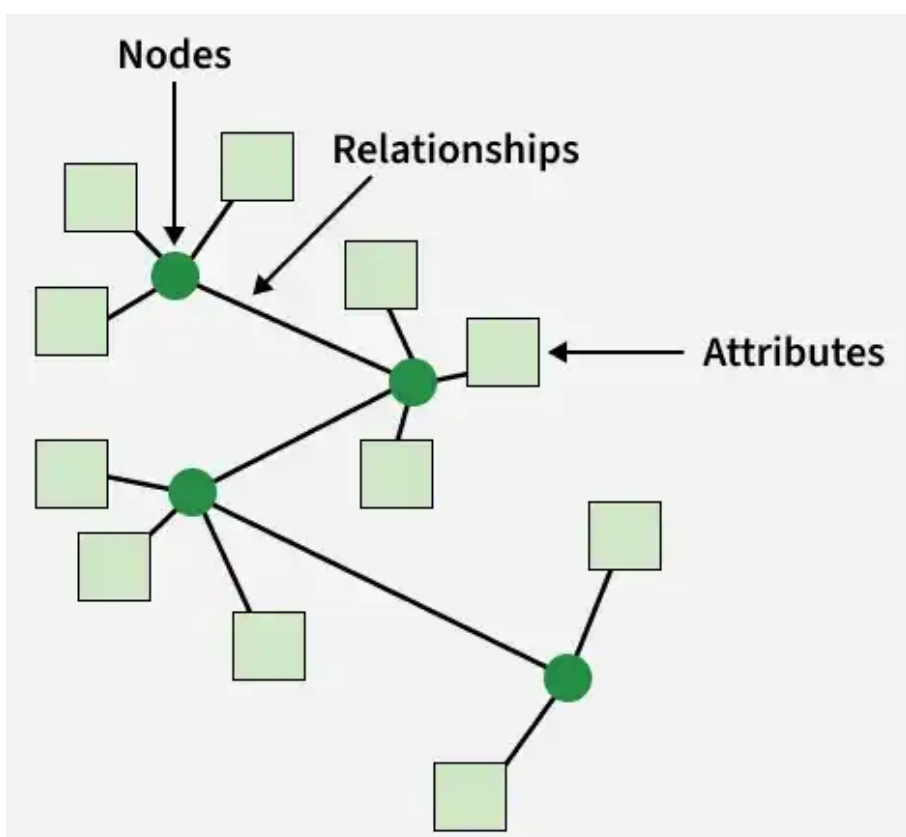
6. Graph Database on NoSQL

A Graph Database is a type of **NoSQL** database that stores data as nodes, relationships (edges) and properties. It is designed to efficiently represent and query connected data.

- Designed to manage interconnected data.
- Uses Neo4j, Amazon Neptune and JanusGraph for graph processing.
- Stores relationships as edges for direct navigation between connected nodes.

6.1 Components of a Graph Database

A graph database stores data as a collection of nodes and relationships instead of tables. The following are the main components of a graph database.



6.1.1 Nodes (Vertices)

Nodes represent entities or objects in the graph. Each node can store properties that describe the entity.

Examples:

- Person
- Customer
- Product
- City

Example

```
(Person: John)
```

6.1.2 Relationships (Edges)

Relationships connect two nodes and represent how they are related. They define the connections between entities.

Examples:

- FRIENDS_WITH
- PURCHASED
- WORKS_FOR
- LOCATED_IN

Example

```
(John) - [:FRIENDS_WITH] -> (Alice)
```

6.1.3 Properties

Properties are key-value pairs that store additional information about nodes and relationships.

Examples:

- Name = "John"
- Age = 25
- Since = 2022

Example:

```
(Person {Name: "John", Age: 25})
```

6.1.4 Labels

Labels categorize nodes into different groups, making it easier to organize and query data.

Examples:

- Person
- Employee
- Customer
- Product

A node can have one or more labels.

6.1.5 Relationship Types

Relationship types define the meaning of the connection between two nodes.

Examples:

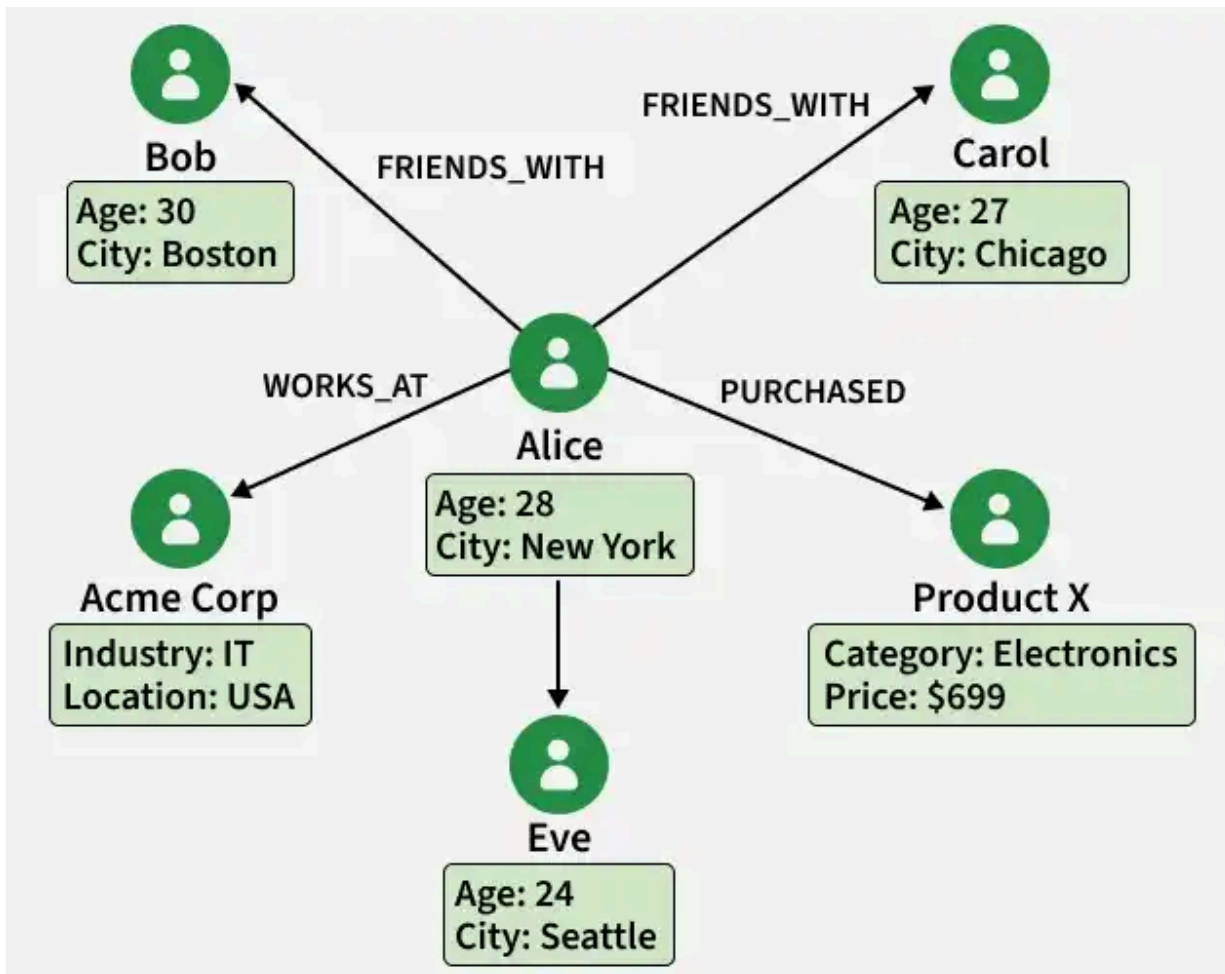
- FRIENDS_WITH

- WORKS_FOR
- PURCHASED
- MANAGES

Using relationship types makes graph traversal more efficient.

6.2 Example of Graph Database

The following graph shows how entities are connected through relationships and enriched with properties in a graph database.



Step 1: Identify the Nodes

Nodes represent the entities in the graph.

In the diagram:

- Alice
- Bob
- Carol
- Eve
- Acme Corp
- Product X

These are all nodes because they represent people, an organization and a product.

Step 2: Identify the Relationships

Relationships connect nodes and describe how they are associated.

In the diagram:

- Alice → FRIENDS_WITH → Bob
- Alice → FRIENDS_WITH → Carol
- Alice → WORKS_AT → Acme Corp
- Alice → PURCHASED → Product X

These relationships directly connect related entities.

Step 3: Identify the Properties

Properties store additional information about nodes.

In the diagram:

- **Alice:** Age = 28, City = New York
- **Bob:** Age = 30, City = Boston
- **Carol:** Age = 27, City = Chicago
- **Eve:** Age = 24, City = Seattle
- **Acme Corp:** Industry = IT, Location = USA
- **Product X:** Category = Electronics, Price = \$699

Step 4: Traverse the Graph

When a query is executed, the graph database follows relationships directly to retrieve connected data instead of performing JOIN operations.

Example:

- To find Alice's friends, the database follows the FRIENDS_WITH relationships and returns Bob and Carol.
- To find where Alice works, it follows the WORKS_AT relationship and returns Acme Corp.
- To find what Alice purchased, it follows the PURCHASED relationship and returns Product X.

6.3 Uses of Graph Databases

Graph databases are widely used in applications where relationships between data are important.

- **Social Networks:** Manage user connections and friendships.
- **Recommendation Systems:** Suggest products, movies or friends based on user preferences.
- **Fraud Detection:** Detect suspicious connections and fraudulent activities.
- **Knowledge Graphs:** Organize connected information for better search and analysis.

6.4 Advantages of Graph Databases

- **High Performance:** Provides fast query execution for relationship-based and connected data.
- **Efficient for Connected Data:** Handles complex relationships effectively, making it ideal for networked data.
- **Flexible Data Model:** Allows easy addition of new nodes and relationships without changing the schema.

6.5 Limitation of Graph Databases

- **Database-Specific Query Languages:** Many graph databases use proprietary query languages, reducing portability.
- **Not Suitable for Transaction-Heavy Applications:** Less efficient for applications with a large number of simple transactions.
- **Requires Specialized Knowledge:** Developers need to understand graph concepts and traversal techniques.